Lesson 8: Under Pressure

Unreal Engine (optional advanced track)

VIDEO GAME CREATION

This week: give the enemies you dodge last week actual stakes. An enemy reaching the bottom now costs a life - lose all 3, and "Game Over" appears.

Open your own [YourName]Game project from Lesson 7 - don't start a new one.

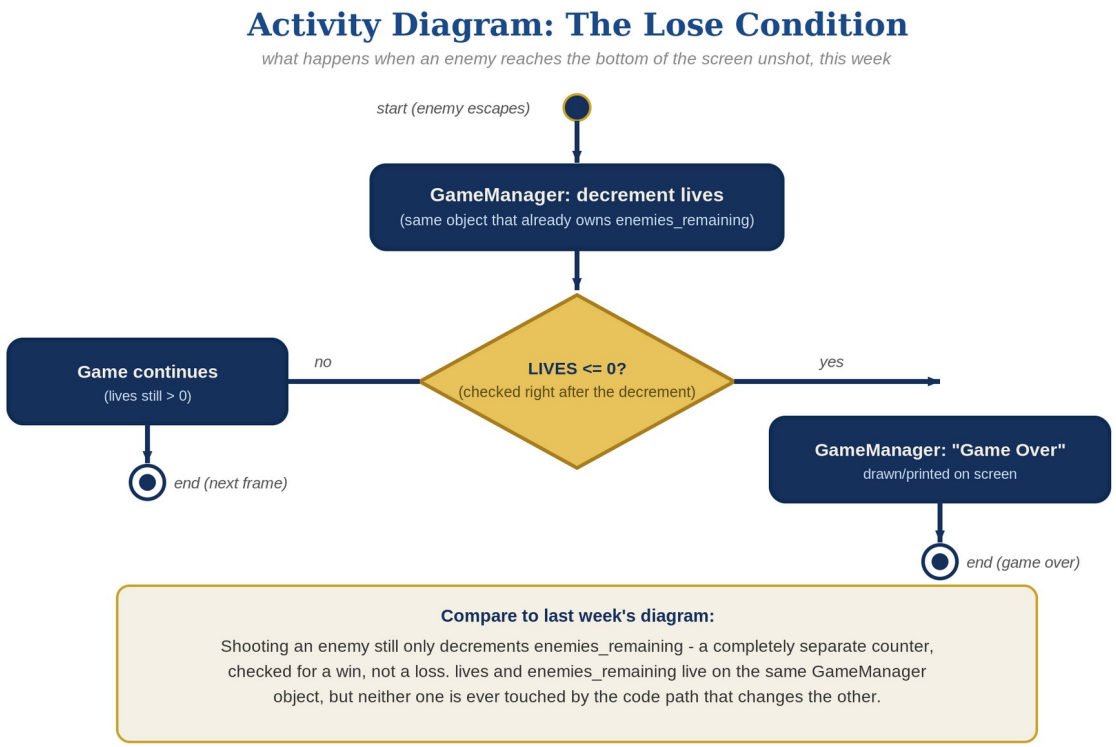
Tip: the core exercise needs no new assets - this reuses last week's escape-cleanup code and just gives it a real consequence. For Level 3 sound, use the provided sfx_lifelost.wav and sfx_gameover.wav from the Lesson 8 Assets folder.

Setup

1. Open your [YourName]Game project from Lesson 7.
2. Open BP_Enemy and add a second Event Dispatcher (Details panel under My Blueprint > Event Dispatchers, click +) named OnEnemyEscaped.
3. In BP_GameManager - the placed Actor introduced in Lesson 6 that already holds EnemiesRemaining - add a second Integer variable Lives, default value 3.

Level 1 - Minimal (MVP)

Every enemy that reaches the bottom unshot now costs the player one life, tracked starting at 3. The moment lives reach zero, "Game Over" appears on screen.



The lose-condition logic behind this week's exercise - the same shape in every engine, whatever the exact code or blocks look like.

4. In BP_Enemy's Event Graph, find the escape Branch you built last week. Right before its Destroy Actor node, add a "Call OnEnemyEscaped" node (code below).
5. In the Level Blueprint's spawn For Loop, right where you bind OnEnemyDestroyed to your BP_GameManager reference, also add a "Bind Event to OnEnemyEscaped" node targeting the same spawned Actor, wired to the same BP_GameManager reference's new HandleEnemyEscaped custom event.
6. In BP_GameManager, build HandleEnemyEscaped: decrease Lives by 1, Branch on Lives <= 0, and on True, add a Print String node with text "Game Over" and "Print to Screen" checked.
7. Click Compile, then Play and let all 3 lives run out by letting enemies escape unshot - "Game Over" should appear the instant the third one escapes.

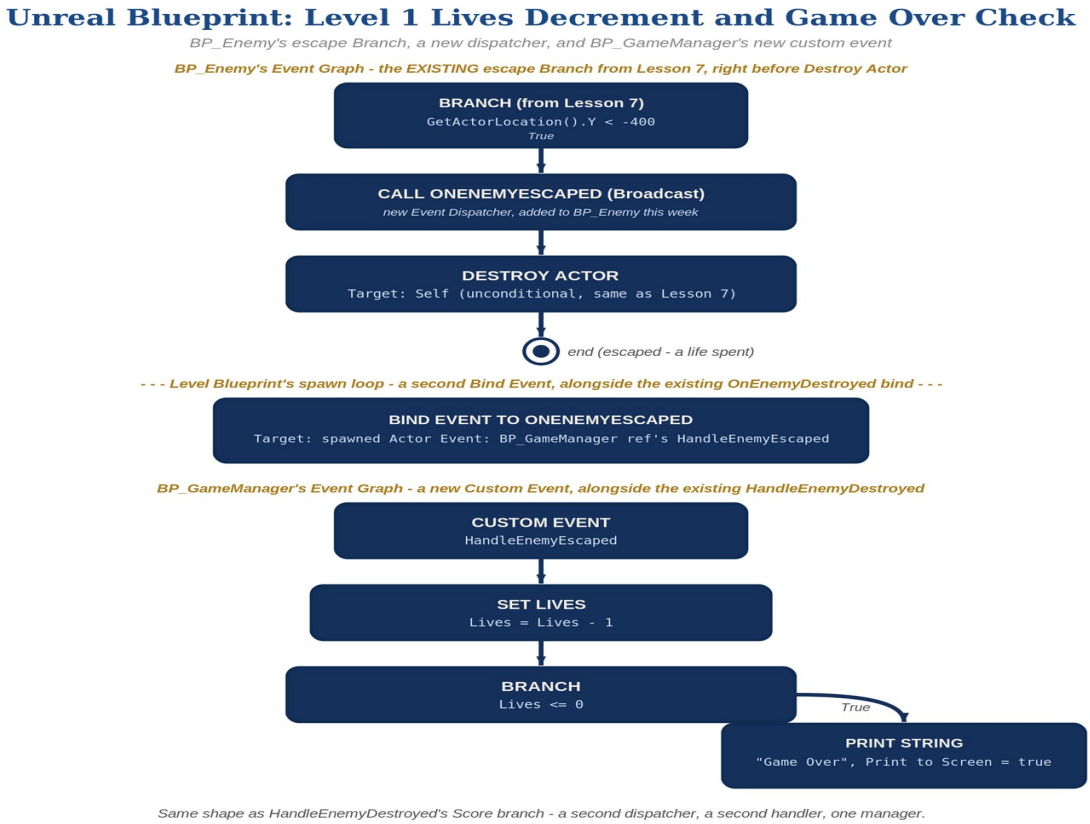
Highlighted lines are new this week; everything else is code you already have.

```
BP_Enemy's Event Graph (escape Branch, revised):
Event Tick ->
  Branch(GetActorLocation().Y < -400) -> True:
    Call OnEnemyEscaped (Broadcast)
    Destroy Actor(Self)

Level Blueprint spawn loop (alongside binding OnEnemyDestroyed to the same
BP_GameManager reference):
  Bind Event to OnEnemyEscaped(Target = spawned Actor, Event =
BP_GameManager ref's HandleEnemyEscaped)

BP_GameManager:
Custom Event HandleEnemyEscaped ->
```

```
  Lives = Lives - 1
  Branch(Lives <= 0) -> True: Print String("Game Over", Print to Screen =
true)
```



BP_Enemy's escape Branch, a new dispatcher, and BP_GameManager's new custom event, in Unreal Blueprint - the node-for-node equivalent of the Level 1 code.

Alternate Path: C++ (code, optional advanced track)

The same escape-vs-hit-dispatcher pattern works in C++ instead of Blueprint nodes.

8. In ACppEnemy.h, declare a second dispatcher-equivalent delegate, e.g. DECLARE_MULTICAST_DELEGATE(FOnEnemyEscaped), alongside the existing OnEnemyDestroyed delegate.
9. In ACppEnemy::Tick()'s escape branch, call OnEnemyEscaped.Broadcast() right before Destroy(), mirroring how OnEnemyDestroyed.Broadcast() is called from OnOverlap().

```
// ACppEnemy.cpp - Tick()'s escape branch (revised):
if (GetActorLocation().Y < -400.f) {
    OnEnemyEscaped.Broadcast();
    Destroy();
}
```

This mirrors the Blueprint path's escape Branch - same underlying broadcast-then-destroy shape, just from code.

Level 2 - Complete (Alpha/Beta)

Shooting an enemy never costs a life, and letting one escape never counts as a kill - the two counters stay completely independent.

- Shoot an enemy and confirm Lives does NOT change - only OnEnemyDestroyed, and EnemiesRemaining, should be involved.
- Let an enemy escape and confirm EnemiesRemaining does NOT change - only OnEnemyEscaped, and Lives, should be involved.
- Confirm "Game Over" prints exactly once, exactly when Lives reaches zero.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Visual: add a Print String in HandleEnemyEscaped showing the current Lives value every time it changes - a nice touch, though the full UMG-based UI is coming properly in Lesson 9.
- Sound: import the provided sfx_lifelost.wav and sfx_gameover.wav as Sound Wave assets, then add a Play Sound 2D node in HandleEnemyEscaped - sfx_lifelost.wav right after Lives decreases, sfx_gameover.wav on the Branch's True output before the Print String node.
- Structure: OnEnemyDestroyed and OnEnemyEscaped are two separate dispatchers, each broadcast from exactly one place in BP_Enemy, each bound to its own handler on BP_GameManager - the same separation pattern used since Lesson 7, just doubled, and both handlers still living on the one dedicated manager Actor introduced in Lesson 6.

If Something's Wrong

- Lives decreases when you shoot an enemy: confirm OnEnemyEscaped is only broadcast from the escape Branch, never from the successful-Cast (hit) branch.
- "Game Over" never prints: confirm the Level Blueprint's spawn loop actually binds OnEnemyEscaped for every spawned Actor, targeting BP_GameManager's HandleEnemyEscaped - not just OnEnemyDestroyed, and not a Custom Event on the Level Blueprint itself.
- Both messages print at the same screen position: adjust HandleEnemyEscaped's Print String Text Position/Duration slightly so it doesn't overlap the win message.

Want More?

dev.epicgames.com/community/learning, Student Resource List > Unreal